

Déployer une application Angular avec GitHub Actions

CI/CD de bout en bout : build, image Docker, registry & déploiement
SSH sur serveur

Formation DevOps • avec illustrations

Formateur : Haithem Mihoubi

8 modules • pipeline complet expliqué pas à pas • 2026

Table des matières

00	Introduction : CI/CD & GitHub Actions	3
01	Anatomie de GitHub Actions	5
02	Préparer l'application Angular pour la production	9
03	Préparer le serveur et les accès	12
04	Le workflow CI/CD complet	15
05	Le déploiement SSH sur le serveur	21
06	Secrets, registry & sécurité	25
07	Aller plus loin, dépanner & checklist	28

Introduction : CI/CD & GitHub Actions

1. Le problème : déployer « à la main »

Déployer une application Angular sur un serveur **manuellement** ressemble souvent à ça :

1. On compile localement (`ng build`).
2. On copie les fichiers via FTP ou `scp` .
3. On se connecte en SSH au serveur.
4. On redémarre Nginx... en croisant les doigts.

Cette approche est **lente, non reproductible et risquée** : un oubli d'étape, une mauvaise version compilée, une variable d'environnement différente, et c'est la panne en production. Surtout, **une seule personne** sait comment déployer.

2. La solution : l'intégration et le déploiement continus (CI/CD)

Sigle	Signification	Ce que ça automatise
CI	<i>Continuous Integration</i>	À chaque commit : installer, builder , tester le code
CD	<i>Continuous Delivery / Deployment</i>	Packager (image Docker), publier , déployer automatiquement

L'idée centrale : chaque `git push` déclenche une chaîne d'étapes automatiques. Le code part du poste du développeur et arrive **tout seul** en production, testé et packagé de façon **identique à chaque fois**.

3. Pourquoi GitHub Actions ?

GitHub Actions est le moteur de CI/CD **intégré à GitHub**. Aucun serveur d'intégration à installer (pas de Jenkins à maintenir) : tout se déclare dans un simple fichier YAML versionné avec le code.

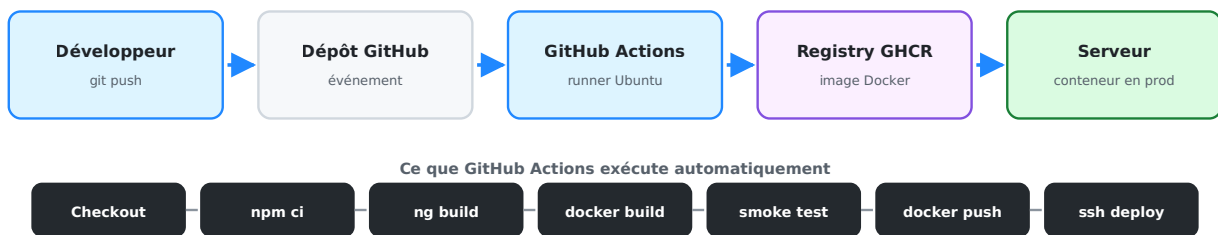
- **Intégré** : déclenché par les événements GitHub (push, pull request...).
- **Gratuit** pour les dépôts publics, généreux pour les privés.
- **Des milliers d'actions réutilisables** sur le Marketplace (`actions/checkout` , `docker/build-push-action` ...).
- **Des runners gérés** : GitHub fournit des machines Ubuntu / Windows / macOS jetables.

- **Secrets intégrés** : clés SSH, tokens, mots de passe stockés de façon chiffrée.

4. La vue d'ensemble de notre pipeline

Dans ce cours, nous construisons le pipeline complet d'une application Angular réelle (**QuickBite Frontend**) : du `git push` jusqu'au conteneur Docker qui tourne sur un serveur.

Le pipeline CI/CD : du commit au serveur en production



Du commit au serveur : GitHub Actions enchaîne build, test, packaging et déploiement.

Le trajet complet :

1. Le développeur fait `git push` sur la branche `main`.
2. GitHub déclenche le **workflow** (événement `push`).
3. Un **runner Ubuntu** compile Angular et construit une **image Docker**.
4. L'image est testée (smoke test HTTP 200) puis **publiée** sur le registry **GHCR**.
5. Le runner se connecte en **SSH** au serveur, qui **télécharge et lance** la nouvelle image.

5. Ce que vous saurez faire à la fin

- Lire et écrire un fichier de workflow `.github/workflows/*.yaml`.
- Comprendre les notions de **workflow**, **job**, **step**, **runner**, **action**, **secret**.
- Préparer une image Docker Angular prête pour la production.
- Configurer un serveur (Docker + accès SSH) et les **secrets** GitHub.
- Écrire un pipeline **build** → **push** → **deploy** complet et sécurisé.
- Dépanner un workflow qui échoue.

Pré-requis : des bases de Git/GitHub, de Docker (voir le cours *Dockeriser ses applications*) et un accès SSH à un serveur Linux.

Anatomie de GitHub Actions

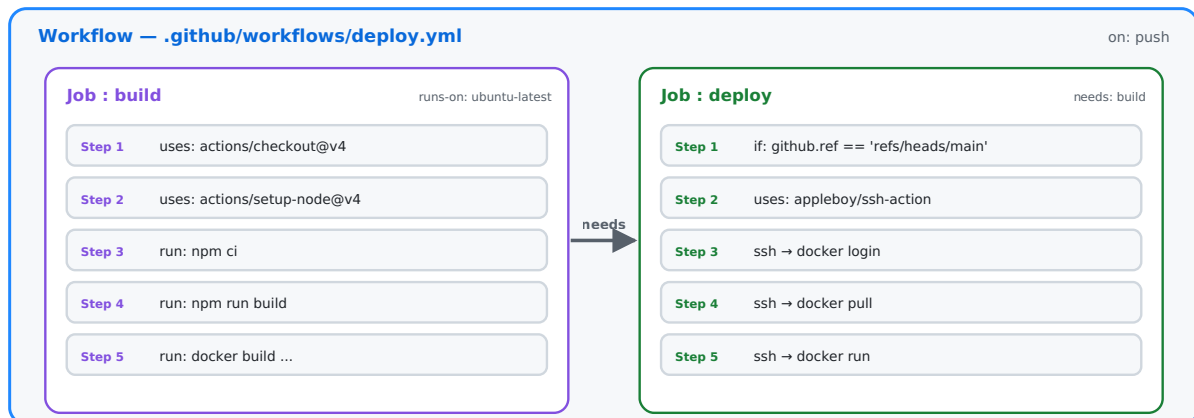
Avant d'écrire une seule ligne de YAML, il faut maîtriser le **vocabulaire**. Tout GitHub Actions repose sur cinq notions emboîtées.

1. Les cinq notions clés

Notion	Définition	Analogie
Workflow	Un fichier <code>.yaml</code> qui décrit un processus automatisé complet	La recette de cuisine
Event (<code>on:</code>)	L'événement qui déclenche le workflow	Le coup de sonnette
Job	Un groupe d'étapes qui s'exécutent sur une même machine	Une équipe en cuisine
Step	Une étape unique : une commande (<code>run</code>) ou une action (<code>uses</code>)	Un geste de la recette
Runner	La machine (VM) jetable qui exécute un job	Le plan de travail

Anatomie d'un workflow GitHub Actions

Workflow → Jobs → Steps — chaque job tourne sur un runner isolé



Un workflow contient des jobs ; chaque job contient des steps et tourne sur son propre runner.

Le point le plus important à retenir

Chaque job démarre sur une machine neuve et isolée. Deux jobs ne partagent ni le disque, ni la mémoire. Si le job `deploy` a besoin de l'image construite par le job `build`, il faut soit la **publier** dans un registry, soit utiliser un **artifact**. C'est pourquoi notre pipeline publie l'image sur GHCR entre les deux.

2. La structure d'un fichier de workflow

Les workflows vivent dans le dossier `.github/workflows/` à la racine du dépôt. Voici le squelette minimal commenté :

```
name: Mon pipeline           # Nom affiché dans l'onglet "Actions"

on:                          # — ÉVÉNEMENT : quand déclencher ?
  push:
    branches: [main]

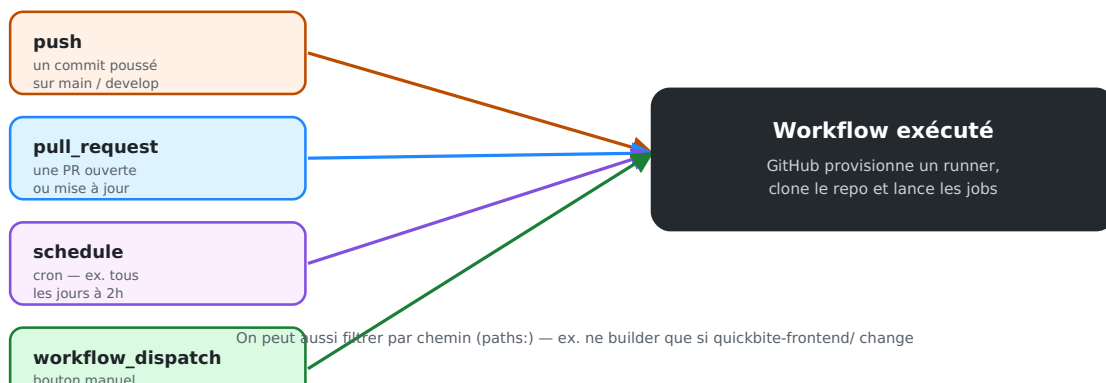
jobs:                         # — JOBS : un ou plusieurs
  build:                     # identifiant du job
    runs-on: ubuntu-latest   # le RUNNER
    steps:                   # — STEPS : les étapes
      - uses: actions/checkout@v4 # une ACTION réutilisable
      - run: echo "Bonjour depuis le CI" # une COMMANDE shell
```

- **uses:** → réutilise une **action** publiée (du Marketplace ou d'un autre dépôt).
Exemple : `actions/checkout@v4` clone votre code sur le runner.
- **run:** → exécute une **commande shell** sur le runner.

3. Les événements déclencheurs (`on:`)

Un workflow ne fait rien tant qu'un **événement** ne survient pas. Les plus courants :

Qu'est-ce qui déclenche un workflow ? (`on:`)



push, pull_request, schedule, workflow_dispatch : autant de portes d'entrée vers le même workflow.

```

on:
  push:
    branches: [main, develop]      # uniquement sur ces branches
    paths:
      - 'quickbite-frontend/**'    # ...et seulement si ce dossier change
  pull_request:
    branches: [main]              # à l'ouverture/màj d'une PR vers main
  schedule:
    - cron: '0 2 * * *'           # tous les jours à 02h00 UTC
  workflow_dispatch:               # bouton "Run workflow" manuel

```

Astuce `paths:` — dans un *monorepo* qui contient le front, le back et la doc, `paths:` évite de relancer tout le pipeline frontend quand seul le backend change. C'est exactement ce que fait notre projet QuickBite.

4. Actions, runners et Marketplace

Les runners

`runs-on: ubuntu-latest` demande à GitHub une **VM Ubuntu jetable**, déjà équipée de Git, Docker, Node, etc. Elle est **créée pour le job puis détruite** — aucune trace, aucune donnée résiduelle. (Pour des besoins spécifiques, on peut héberger ses propres *self-hosted runners*.)

Les actions réutilisables

Une **action** est un composant packagé que l'on branche avec `uses:`. On épingle toujours une **version** (`@v4`) pour la reproductibilité.

Action	Rôle
<code>actions/checkout@v4</code>	Cloner le dépôt sur le runner
<code>actions/setup-node@v4</code>	Installer Node.js (+ cache npm)
<code>docker/setup-buildx-action@v3</code>	Activer le builder Docker avancé
<code>docker/build-push-action@v6</code>	Builder et pousser une image
<code>docker/login-action@v3</code>	S'authentifier sur un registry
<code>appleboy/ssh-action@v1</code>	Exécuter des commandes via SSH

5. Le cycle de vie résumé

1. **Événement** — quelqu'un pousse du code → GitHub détecte `on: push`.
2. **Provisioning** — GitHub alloue un runner neuf par job.

3. **Exécution** — chaque step s'exécute dans l'ordre ; si une step échoue, le job s'arrête (sauf `continue-on-error: true`).
4. **Dépendances** — `needs:` enchaîne les jobs ; sinon ils tournent **en parallèle**.
5. **Résultat** —  ou  visible sur le commit, la PR et l'onglet *Actions*.
6. **Nettoyage** — les runners sont détruits.

Préparer l'application Angular pour la production

GitHub Actions automatise un déploiement, mais encore faut-il que l'application sache **se construire et se servir** de façon reproductible. La brique de base : une **image Docker multi-stage**.

1. Pourquoi Docker plutôt que copier le `dist/` ?

Approche	Problème
Copier <code>dist/</code> via <code>scp</code>	Il faut Node + Nginx déjà configurés sur le serveur, versions qui dérivent
Image Docker	Tout est figé dans l'image : « ça marche sur le runner = ça marche en prod »

Une image Docker rend le déploiement **atomique** (on lance une image, point) et **réversible** (on peut relancer l'ancienne en cas de souci).

2. Le Dockerfile multi-stage

Une application Angular se **compile** avec Node.js, puis se **sert** avec Nginx. On sépare donc en deux étapes : Node ne survit pas en production, ce qui donne une image finale minuscule (~50 Mo au lieu de ~1 Go).

```
# ----- Étape 1 : build -----
FROM node:20-alpine AS build
WORKDIR /app

# Dépendances d'abord → cache de couche Docker
COPY package.json package-lock.json ./
RUN npm ci

# Puis le code et le build de production
COPY . .
RUN npm run build -- --configuration production

# ----- Étape 2 : runtime -----
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Angular 17+ : la sortie est dans dist/<app>/browser
COPY --from=build /app/dist/quickbite-frontend/browser /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Attention au chemin de sortie selon la version d'Angular : - Angular ≤ 16 : `dist/`
`<nom-app>` - Angular **17+** (builder `application`) : `dist/<nom-app>/browser`

3. La configuration Nginx pour une SPA

Indispensable : une application Angular est une **SPA** (Single Page Application). Toutes les routes inconnues doivent retomber sur `index.html`, sinon rafraîchir la page sur `/orders` renvoie une **erreur 404**.

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    # Cache long des assets versionnés (nom avec hash)
    location ~* \.(?:js|css|woff2?|svg|png|jpg|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Routage SPA : tout le reste retombe sur index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

4. Le `.dockerignore`

On évite d'envoyer `node_modules` et `dist` dans le contexte de build (build plus rapide, image plus saine) :

```
node_modules
dist
.angular
.git
Dockerfile
.dockerignore
*.md
```

5. Vérifier en local avant d'automatiser

Règle d'or : si ça ne marche pas en local, ça ne marchera pas dans le CI. On valide l'image manuellement **une fois** :

```
docker build -t quickbite-frontend:test .  
docker run -p 8080:80 quickbite-frontend:test  
# → ouvrir http://localhost:8080 et tester les routes (F5 sur /orders)
```

Une fois cette image fonctionnelle, GitHub Actions ne fera que **rejouer ces mêmes commandes** automatiquement à chaque push. C'est l'objet des modules suivants.

Pour approfondir le multi-stage (Spring Boot, React, Python, multi-environnement), voir le cours dédié « *Dockeriser ses applications* ».

Préparer le serveur et les accès

Le runner GitHub doit pouvoir **se connecter au serveur** et y **lancer Docker**. Ce module prépare le terrain côté serveur, puis enregistre les accès dans les **secrets** GitHub.

1. Installer Docker sur le serveur

Sur un serveur Ubuntu/Debian fraîchement provisionné :

```
# Installation officielle de Docker
curl -fsSL https://get.docker.com | sh

# Démarrer Docker au boot
sudo systemctl enable --now docker

# Vérifier
docker --version
```

2. Créer un utilisateur de déploiement

On évite d'utiliser `root` pour le déploiement. On crée un utilisateur dédié, membre du groupe `docker` (pour lancer des conteneurs sans `sudo`) :

```
sudo adduser --disabled-password deployer
sudo usermod -aG docker deployer
```

Sécurité : un utilisateur dédié limite la casse si la clé fuit, et trace clairement « qui » déploie dans les logs du serveur.

3. Générer une paire de clés SSH dédiée

Le runner s'authentifiera par **clé SSH** (jamais par mot de passe). On génère une paire **spécifique au déploiement** — pas votre clé personnelle.

```
# Sur votre poste (ou le serveur), SANS passphrase pour l'automatisation
ssh-keygen -t ed25519 -C "github-actions-deploy" -f deploy_key
# → produit deux fichiers :
#   deploy_key      (clé PRIVÉE → ira dans les secrets GitHub)
#   deploy_key.pub  (clé PUBLIQUE → ira sur le serveur)
```

Autoriser la clé publique sur le serveur

```
# Ajouter la clé PUBLIQUE aux clés autorisées de l'utilisateur deployer
ssh-copy-id -i deploy_key.pub deployer@VOTRE_SERVEUR
# (ou copier manuellement le contenu dans ~/.ssh/authorized_keys)
```

Tester la connexion avec la clé **privée** :

```
ssh -i deploy_key deployer@VOTRE_SERVEUR "docker ps"
```

Si cette commande liste les conteneurs, le runner pourra faire de même.

4. Enregistrer les secrets dans GitHub

Règle absolue : aucune clé, aucun mot de passe, aucune IP sensible dans le code. Tout passe par les **secrets chiffrés** de GitHub.

Dans le dépôt GitHub : **Settings** ► **Secrets and variables** ► **Actions** ► **New repository secret**.

Secret	Contenu	Exemple
DEPLOY_HOST	IP ou domaine du serveur	203.0.113.10
DEPLOY_USER	utilisateur SSH	deployer
DEPLOY_SSH_KEY	contenu du fichier <code>deploy_key</code> (clé privée)	-----BEGIN OPENSSH...
DEPLOY_PORT	port SSH (optionnel, défaut 22)	22

Pour copier la clé privée intégralement dans le presse-papier :

```
cat deploy_key      # copier TOUT, y compris les lignes BEGIN/END
```

GITHUB_TOKEN : **pas besoin de le créer**. GitHub injecte automatiquement ce secret dans chaque workflow. Il sert à s'authentifier sur le registry **GHCR** (voir module 06).

5. Checklist avant d'écrire le workflow

- [] Docker installé et actif sur le serveur (`docker ps` fonctionne).
- [] Utilisateur `deployer` créé et membre du groupe `docker` .
- [] Paire de clés `deploy_key` / `deploy_key.pub` générée.
- [] Clé publique ajoutée à `authorized_keys` sur le serveur.

- [] `ssh -i deploy_key deployer@serveur "docker ps"` réussit.
- [] Secrets `DEPLOY_HOST` , `DEPLOY_USER` , `DEPLOY_SSH_KEY` enregistrés sur GitHub.

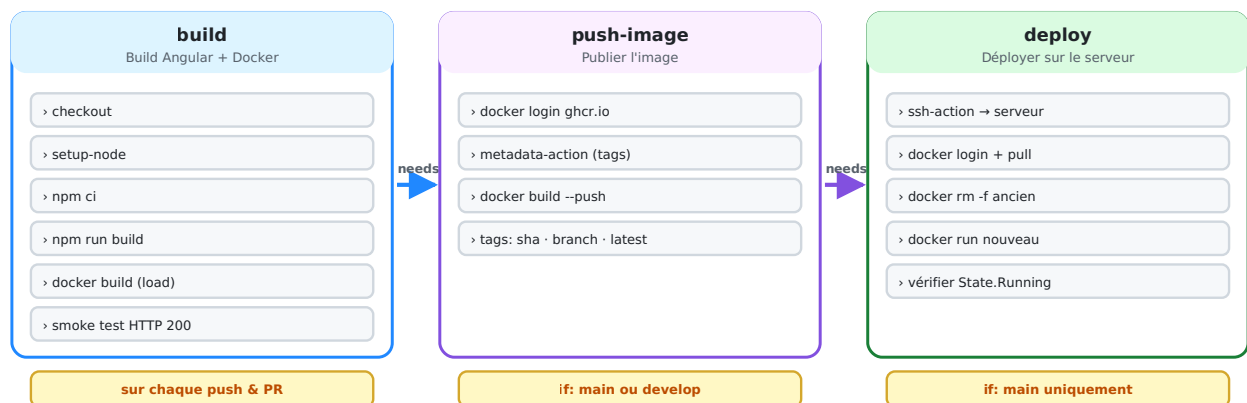
Tout est prêt : on peut maintenant écrire le pipeline complet.

Le workflow CI/CD complet

Nous assemblons maintenant tout : un pipeline en **trois jobs enchaînés** qui builde, publie et déploie l'application. C'est le workflow réel du projet **QuickBite**.

1. La stratégie en trois jobs

Le pipeline en 3 jobs enchaînés (needs)



build (sur chaque push/PR) → push-image (main/develop) → deploy (main seulement), reliés par needs.

Job	Quand ?	Rôle
build	chaque <code>push</code> et chaque <code>pull_request</code>	Compiler Angular, builder l'image, smoke test
push-image	seulement sur <code>main</code> ou <code>develop</code>	Publier l'image taguée sur GHCR
deploy	seulement sur <code>main</code>	Se connecter en SSH et lancer l'image sur le serveur

Pourquoi séparer ? On veut **valider chaque PR** (job `build`) sans pour autant publier ni déployer. Seuls les vrais changements fusionnés sur `main` vont en production. Les conditions `if:` réalisent ce filtrage.

2. L'en-tête : déclencheurs et variables

```
name: QuickBite Frontend – CI/CD

on:
  push:
    branches: [main, develop]
    paths:
      - 'quickbite-frontend/**' # ne builder que si le front change
      - '.github/workflows/frontend-deploy.yml'
  pull_request:
    branches: [main]
    paths:
      - 'quickbite-frontend/**'

env: # variables réutilisées partout
  REGISTRY: ghcr.io
  IMAGE_NAME: quickbite-frontend
  DOCKERFILE_PATH: quickbite-frontend/Dockerfile
  CONTEXT_PATH: quickbite-frontend
```

Le bloc `env:` au niveau du workflow définit des variables disponibles dans tous les jobs via `${{ env.NOM }}` — pas de répétition, un seul endroit à modifier.

3. Job 1 — Build, image Docker & smoke test

```
jobs:
  build:
    name: Build Angular + Docker
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm' # met en cache ~/.npm
          cache-dependency-path: ${ env.CONTEXT_PATH }}/package-lock.json

      - name: Install dependencies
        run: npm ci
        working-directory: ${ env.CONTEXT_PATH }

      - name: Build Angular (production)
        run: npm run build
        working-directory: ${ env.CONTEXT_PATH }

      - name: Setup Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build Docker image (sans push)
        uses: docker/build-push-action@v6
        with:
          context: ${ env.CONTEXT_PATH }
          file: ${ env.DOCKERFILE_PATH }
          tags: ${ env.IMAGE_NAME }:${ env.github.sha }
          load: true # charge l'image localement
          cache-from: type=gha # réutilise le cache GitHub Actions
          cache-to: type=gha,mode=max
```

Le smoke test : vérifier que l'image démarre vraiment

On ne se contente pas de builder : on **lance le conteneur** et on vérifie qu'il répond en HTTP 200. Un build qui compile mais ne démarre pas serait inutile.

```
- name: Smoke test – démarrer le conteneur et vérifier HTTP 200
run: |
  docker run -d -p 8080:80 --name smoke-test ${env.IMAGE_NAME}:${github.sha}
  sleep 3
  STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/)
  docker logs smoke-test
  docker rm -f smoke-test
  if [ "$STATUS" != "200" ]; then
    echo "Smoke test failed: HTTP $STATUS"
    exit 1 # fait échouer le job → bloque le déploiement
  fi
  echo "Smoke test passed: HTTP $STATUS"
```

`${github.sha}` est le hash du commit. Taguer l'image avec lui rend **chaque build traçable** : on sait exactement quel commit tourne en production.

4. Job 2 — Publier l'image sur le registry

```
push-image:
  name: Push image to Registry
  needs: build # n'exécute QUE si "build" a réussi
  if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4
    - uses: docker/setup-buildx-action@v3

    - name: Log in to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN } # token fourni automatiquement

    - name: Extraire les métadonnées (tags, labels)
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ env.REGISTRY }/${ github.repository }/${ env.IMAGE_NAME }
        tags: |
          type=sha,prefix=,suffix=,format=short # tag = hash court du commit
          type=ref,event=branch # tag = nom de la branche
          type=raw,value=latest,enable=${ github.ref == 'refs/heads/main' }

    - name: Build & Push image
      uses: docker/build-push-action@v6
      with:
        context: ${ env.CONTEXT_PATH }
        file: ${ env.DOCKERFILE_PATH }
        push: true # cette fois on POUSSE
        tags: ${ steps.meta.outputs.tags }
        labels: ${ steps.meta.outputs.labels }
        cache-from: type=gha
        cache-to: type=gha,mode=max
```

- `needs: build` crée la dépendance : `push-image` attend la réussite de `build`.
- `steps.meta.outputs.tags` récupère la sortie de la step `id: meta` — c'est ainsi qu'on passe une valeur d'une step à une autre.

5. Job 3 — Déploiement (aperçu)

Le troisième job se connecte au serveur en SSH pour y lancer la nouvelle image. Il est suffisamment important pour avoir **son propre module** : voir le module suivant.

```
deploy:
  name: Deploy on server
  needs: push-image
  if: github.ref == 'refs/heads/main'           # production = main uniquement
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Déploiement via SSH
      uses: appleboy/ssh-action@v1.2.2
      with:
        host: ${ secrets.DEPLOY_HOST }
        username: ${ secrets.DEPLOY_USER }
        key: ${ secrets.DEPLOY_SSH_KEY }
        script: |
          # ... pull + run de l'image (détaillé au module 05)
```

6. Où placer le fichier ?

```
mon-projet/
├── .github/
│   └── workflows/
│       └── frontend-deploy.yml ← le workflow complet
```

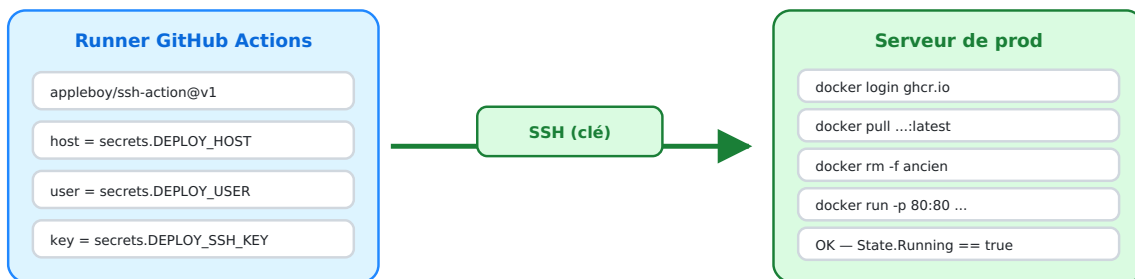
Dès le prochain `git push` sur `main`, ouvrez l'onglet **Actions** du dépôt : vous verrez les trois jobs s'enchaîner en direct.

Le déploiement SSH sur le serveur

Le job `deploy` est le maillon final : il transforme une image publiée en un **conteneur qui tourne en production**. Tout se joue à travers une connexion SSH.

1. Le principe

Le déploiement SSH : du runner GitHub au serveur



La clé privée vit dans les Secrets GitHub, jamais dans le code. Le runner s'auto-détruit après le job.

Le runner ouvre un tunnel SSH (authenticifié par clé), puis le serveur télécharge et lance l'image.

Le runner GitHub **ne déploie pas lui-même** : il **donne des ordres** au serveur via SSH. C'est le serveur qui télécharge l'image depuis GHCR et lance le conteneur. Le runner, lui, est détruit aussitôt après.

2. Le job complet

```
deploy:
  name: Deploy on server
  needs: push-image
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  timeout-minutes: 10

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Déploiement via SSH
      uses: appleboy/ssh-action@v1.2.2
      with:
        host: ${ secrets.DEPLOY_HOST }
        username: ${ secrets.DEPLOY_USER }
        key: ${ secrets.DEPLOY_SSH_KEY }
        port: ${ secrets.DEPLOY_PORT || '22' }
        script: |
          set -e          # arrête le script à la première erreur
          # ... voir détail ci-dessous
```

- `appleboy/ssh-action` ouvre la connexion SSH avec les secrets et exécute le `script:` sur le serveur distant.
- `${ secrets.DEPLOY_PORT || '22' }` — l'opérateur `||` fournit une valeur par défaut si le secret n'est pas défini.
- `set -e` garantit qu'une commande qui échoue **fait échouer tout le déploiement**.

3. Le script exécuté sur le serveur

```
set -e

# 1) S'authentifier sur le registry GHCR
echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${github.actor} --password-stdin

# 2) Télécharger la dernière image
docker pull ${env.REGISTRY}/${github.repository}/${env.IMAGE_NAME}:latest

# 3) Arrêter et supprimer l'ancien conteneur (sans planter s'il n'existe pas)
docker rm -f quickbite-frontend 2>/dev/null || true

# 4) Lancer le nouveau conteneur
docker run -d \
  --name quickbite-frontend \
  --restart unless-stopped \
  -p 80:80 \
  -p 443:443 \
  ${env.REGISTRY}/${github.repository}/${env.IMAGE_NAME}:latest

# 5) Nettoyer les anciennes images (libère le disque)
docker image prune -af

# 6) Vérifier que le conteneur tourne réellement
sleep 3
if [ "$(docker inspect -f '{{.State.Running}}' quickbite-frontend)" != "true" ]; then
  echo "Le conteneur ne tourne pas !"
  docker logs quickbite-frontend
  exit 1
fi
echo "Déploiement réussi !"
```

Les six étapes décortiquées

#	Étape	Pourquoi
1	<code>docker login ghcr.io</code>	L'image est privée par défaut ; il faut s'authentifier pour la <code>pull</code>
2	<code>docker pull</code>	Récupère la dernière version publiée par le job précédent
3	<code>docker rm -f</code>	Libère le nom et les ports ; <code> true</code> évite l'échec si absent
4	<code>docker run</code>	Démarre la nouvelle version
5	<code>docker image prune</code>	Évite la saturation du disque par les vieilles images
6	<code>docker inspect</code>	Vérification finale : le job échoue si le conteneur ne tourne pas

4. Les options clés de `docker run`

- `-d` : mode détaché (le conteneur tourne en arrière-plan).

- `--restart unless-stopped` : le conteneur **redémarre tout seul** si le serveur reboote ou si le processus plante. Essentiel en production.
- `-p 80:80 -p 443:443` : expose les ports HTTP/HTTPS du serveur vers le conteneur.
- `--name` : un nom fixe pour pouvoir le retrouver, l'arrêter et le remplacer.

5. Stratégie de déploiement : « stop puis start »

Ce script applique un déploiement simple : on **arrête** l'ancien conteneur puis on **démarre** le nouveau. Il y a une **micro-coupure** (quelques secondes) le temps du redémarrage.

Stratégie	Coupure	Complexité
Stop & Start (ce cours)	quelques secondes	très simple ✓
Blue-Green	zéro	il faut un reverse-proxy qui bascule
Rolling (orchestrateur)	zéro	nécessite Docker Swarm / Kubernetes

Pour un site vitrine ou une app interne, *stop & start* est parfaitement acceptable. Le **healthcheck** de l'étape 6 garantit qu'on détecte immédiatement un démarrage raté.

6. Vérifier le déploiement

Après un run réussi, sur le serveur :

```
docker ps                # le conteneur doit être "Up"
curl -I http://localhost  # doit répondre HTTP/1.1 200 OK
docker logs quickbite-frontend --tail 20  # logs Nginx
```

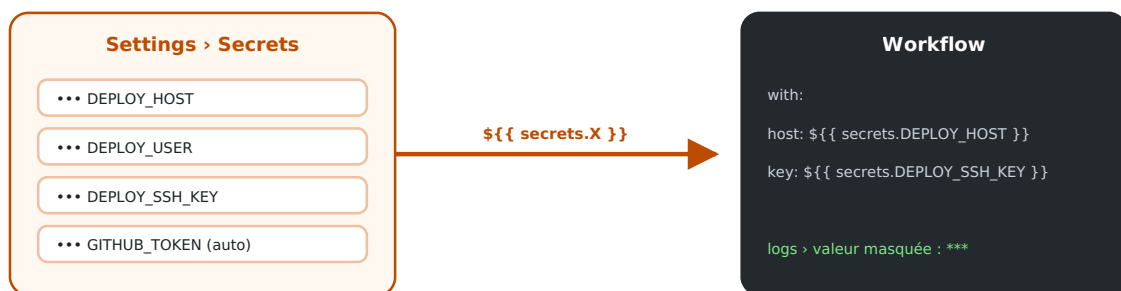
Et bien sûr : ouvrir le domaine du serveur dans un navigateur. Le nouveau code est en ligne — **sans aucune intervention manuelle**.

Secrets, registry & sécurité

Un pipeline manipule des **clés SSH**, des **tokens** et des **identifiants de registry**. Ce module explique comment GitHub les protège et comment publier proprement sur GHCR.

1. Comment fonctionnent les secrets

Les secrets : injectés au runtime, jamais exposés



Dans les logs, GitHub remplace automatiquement toute valeur de secret par ***

Les secrets sont stockés chiffrés, injectés dans le runner à l'exécution, et masqués (***) dans les logs.

Principe	Détail
Chiffrés au repos	Stockés chiffrés dans GitHub ; illisibles une fois enregistrés
Injectés au runtime	Disponibles seulement pendant le job, via <code>{{ secrets.X }}</code>
Masqués dans les logs	Toute valeur de secret affichée devient <code>***</code> automatiquement
Cloisonnés	Non transmis aux workflows déclenchés par des PR de <i>forks</i> externes

```
with:
  host: {{ secrets.DEPLOY_HOST }}
  key: {{ secrets.DEPLOY_SSH_KEY }}
# Dans les logs : host: ***    key: ***
```

À ne jamais faire : écrire une clé ou un mot de passe en clair dans le YAML, ou faire `echo "$MA_CLE"` pour « déboguer ». Si un secret se retrouve dans l'historique Git, considérez-le **compromis** et **révoquez-le** immédiatement.

2. `GITHUB_TOKEN` : le secret automatique

GitHub crée **automatiquement** un secret `GITHUB_TOKEN` pour chaque exécution de workflow. Inutile de le générer ou de le stocker.

- Il s'authentifie auprès de l'API GitHub et du registry **GHCR** du dépôt.
- Il est **éphémère** : valable uniquement le temps du job, puis révoqué.
- Ses permissions se règlent finement avec le bloc `permissions:`.

```
permissions:
  contents: read      # lire le code
  packages: write     # publier des images sur GHCR
```

Principe du moindre privilège : ne donnez que les permissions nécessaires. Pour publier une image, `packages: write` suffit.

3. Publier sur GHCR (GitHub Container Registry)

GHCR (`ghcr.io`) est le registry d'images Docker intégré à GitHub. L'image est rangée « à côté » du code, avec les mêmes droits d'accès.

```
- name: Log in to GitHub Container Registry
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${GITHUB_ACTOR} # l'auteur du push
    password: ${GITHUB_TOKEN} # le token automatique
```

L'image publiée s'appelle :

```
ghcr.io/<organisation>/<depot>/quickbite-frontend:<tag>
```

Bien taguer ses images

`docker/metadata-action` génère plusieurs tags d'un coup :

Tag	Exemple	Usage
hash du commit	<code>a1b2c3d</code>	traçabilité : revenir précisément à une version
nom de branche	<code>main</code> , <code>develop</code>	dernier état d'une branche
<code>latest</code>	<code>latest</code>	la dernière prod (uniquement depuis <code>main</code>)

Bonne pratique : déployez de préférence par **hash de commit** plutôt que `latest`. `latest` est pratique mais ambigu (« quelle version exactement ? ») ; le hash, lui, est sans équivoque et permet un *rollback* fiable.

4. Variables vs secrets

GitHub distingue deux types de configuration (**Settings ► Secrets and variables**) :

	Secrets	Variables
Pour	clés, tokens, mots de passe	config non sensible
Visible	non, masqué <code>***</code>	oui, en clair dans les logs
Accès	<code>\${{ secrets.X }}</code>	<code>\${{ vars.X }}</code>
Exemple	<code>DEPLOY_SSH_KEY</code>	<code>NODE_VERSION</code> , <code>API_URL</code> publique

5. Les environnements protégés (`environment:`)

Pour la production, on peut exiger une **validation manuelle** avant le déploiement. On crée un *environment* (**Settings ► Environments**) avec des **required reviewers**, puis :

```
deploy:
  environment: production    # déclenche une approbation manuelle
  runs-on: ubuntu-latest
  steps:
    - ...
```

Le job `deploy` reste alors **en attente** jusqu'à ce qu'un relecteur autorisé clique sur « Approve ». Idéal pour garder un humain dans la boucle avant la prod.

6. Checklist sécurité

- [] Aucun secret en clair dans le code ni dans l'historique Git.
- [] Clé SSH **dédiée** au déploiement (révocable indépendamment).
- [] `permissions:` réduites au strict nécessaire.
- [] Déploiement par **hash de commit** pour un rollback fiable.
- [] Production protégée par un **environment** avec approbation (si critique).
- [] Images privées sur GHCR sauf besoin explicite de les rendre publiques.

Aller plus loin, dépanner & checklist

Le pipeline fonctionne. Voici les techniques pour l'optimiser, les pannes courantes et une checklist finale.

1. Accélérer le pipeline : le cache

Réinstaller `node_modules` et rebuild l'image à chaque run est lent. Deux caches combinés divisent le temps par deux ou trois.

```
# Cache des dépendances npm
- uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: 'npm'

# Cache des couches Docker (réutilisé entre les runs)
- uses: docker/build-push-action@v6
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

2. Tester plusieurs versions : les matrices

Une **matrice** rejoue le même job sur plusieurs configurations, **en parallèle** :

```
strategy:
  matrix:
    node-version: [18, 20, 22]
steps:
- uses: actions/setup-node@v4
  with:
    node-version: ${ matrix.node-version }
- run: npm ci && npm test
```

GitHub lance ici **trois jobs simultanés**, un par version de Node.

3. Ne pas bloquer sur les étapes non critiques

```
- name: Lint
  run: npx ng lint
  continue-on-error: true      # un warning de lint ne bloque pas le déploiement
```

`continue-on-error: true` laisse le job continuer même si la step échoue — à réserver aux étapes **informatives**, jamais aux tests qui garantissent la qualité.

4. Les expressions et contextes utiles

Expression	Donne
<code>\${{ github.sha }}</code>	hash du commit
<code>\${{ github.ref }}</code>	référence (<code>refs/heads/main</code>)
<code>\${{ github.actor }}</code>	auteur du déclenchement
<code>\${{ github.repository }}</code>	organisation/depot
<code>\${{ github.event_name }}</code>	<code>push</code> , <code>pull_request</code> ...
<code>\${{ secrets.X }}</code> / <code>\${{ vars.X }}</code>	secret / variable

Conditions fréquentes :

```
if: github.ref == 'refs/heads/main'           # seulement sur main
if: github.event_name == 'push'               # seulement sur push, pas PR
if: success() && github.ref == 'refs/heads/main' # combiné
```

5. Notifier l'équipe

On ajoute une notification finale (Slack, Discord, e-mail) conditionnée au résultat :

```
- name: Notifier en cas d'échec
  if: failure()
  run: echo "Le déploiement a échoué sur ${{ github.sha }}"
  # ... ou une action Slack/Discord du Marketplace
```

`failure()` , `success()` , `always()` permettent de réagir selon l'issue du job.

6. Dépannage : les erreurs les plus fréquentes

Symptôme	Cause probable	Solution
<code>npm ci</code> échoue	pas de <code>package-lock.json</code>	committer le lockfile
Mauvais dossier copié	chemin <code>dist/</code> erroné	Angular 17+ → <code>dist/<app>/browser</code>
404 en rafraîchissant une route	config SPA Nginx absente	<code>try_files ... /index.html</code>
<code>permission denied (publickey)</code>	clé SSH invalide ou non autorisée	recopier <code>deploy_key</code> , vérifier <code>authorized_keys</code>
<code>denied: permission_denied</code> au push GHCR	permissions du token	ajouter <code>packages: write</code>
<code>docker: command not found</code> (serveur)	Docker non installé	installer Docker, ajouter l'utilisateur au groupe <code>docker</code>
Le job <code>deploy</code> ne se lance pas	condition <code>if:</code> ou <code>needs:</code>	vérifier la branche et la réussite du job précédent
Conteneur redémarre en boucle	erreur applicative	<code>docker logs <nom></code> sur le serveur

Méthode : lire les logs **de bas en haut** dans l'onglet Actions, repérer la **première** step en rouge. C'est presque toujours là qu'est la cause réelle.

7. Bonnes pratiques

- **Épingler les versions d'actions** (`@v4`) — jamais de tag mouvant non maîtrisé.
- **Un workflow = une responsabilité** claire et lisible.
- `timeout-minutes:` sur chaque job pour éviter un runner bloqué qui consomme.
- **Filtrer avec** `paths:` dans un monorepo.
- **Tester en PR, déployer sur main** — la séparation que nous avons mise en place.
- **Tracer par hash de commit** pour des rollbacks fiables.
- **Tout secret dans les Secrets**, jamais dans le code.

8. Checklist finale du pipeline

- [] Le Dockerfile multi-stage build et démarre **en local**.
- [] Le workflow est dans `.github/workflows/`.
- [] `on:` déclenche sur les bonnes branches, filtré par `paths:`.
- [] Job `build` : checkout → setup-node → `npm ci` → build → image → **smoke test**.
- [] Job `push-image` : conditionné `main / develop`, login GHCR, tags via metadata.

- [] Job `deploy` : conditionné `main`, SSH, `pull` + `run` + **healthcheck**.
 - [] Secrets `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY` configurés.
 - [] Caches npm et Docker activés.
 - [] `--restart unless-stopped` sur le conteneur de prod.
 - [] (Optionnel) Environnement `production` avec approbation manuelle.
-

Conclusion

Vous disposez d'un pipeline **CI/CD complet** : chaque `git push` sur `main` compile l'application Angular, construit et teste une image Docker, la publie sur GHCR, puis la déploie automatiquement sur votre serveur via SSH — **de façon reproductible, tracée et sécurisée**.

Le déploiement n'est plus un rituel manuel stressant : c'est un **effet de bord du merge**. C'est tout l'intérêt du DevOps.

Pour aller plus loin : reverse-proxy Traefik/Nginx avec HTTPS automatique (Let's Encrypt), déploiement *blue-green* sans coupure, orchestration avec Docker Swarm ou Kubernetes, et tests end-to-end (Cypress/Playwright) intégrés au job `build`.